SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Darani E (192312120) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Darani E

Annexure A

Analytical Problem Solving

1.Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer:

Problem: Given a 4-gallon jug and a 3-gallon jug without measuring marks, obtain exactly 2 gallons of water in the 4-gallon jug using a pump, pouring between jugs, and emptying water onto the ground.

Solution:

Let:

- Jug A = 4-gallon jug
- Jug B = 3-gallon jug
- Initial state = (0, 0)

The state represents the amount of water in Jug A and Jug B, respectively.

Step	Action	Jug A (4 gallons)	Jug B (3 gallons)
1	Fill Jug B completely	0	3
2	Pour Jug B into Jug A	3	0
3	Fill Jug B completely again	3	3
4	Pour water from Jug B into Jug A until Jug A is full	4	2
5	Empty Jug A onto the ground	0	2
6	Pour the remaining 2 gallons from Jug B into Jug A	2	0

Final result: Jug A contains exactly 2 gallons of water.

Conclusion: The water jug problem is solved by applying a sequence of actions to move from the initial state to the goal state.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. What are the percept's for this agent?
ii. Characterize the operating environment.
iii. What are the actions the agent can take?
iv. How can one evaluate the performance of the agent?
v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer:

A Mars Rover is an intelligent robotic agent designed to explore the surface of Mars, collect scientific data, analyse soil and rock samples, and transmit information to Earth.

1) What are the percepts for this agent?

Percepts are the information received by an agent from its sensors.

The Mars Rover receives the following percepts:

- Camera images: Images of rocks, soil, and the surrounding terrain.
- Temperature sensors: Surface and environmental temperature.
- Chemical sensors: Chemical composition of soil and rock samples.
- Navigation sensors: Position, orientation, and movement.
- Obstacle sensors: Rocks, craters, and other hazards.
- Scientific instruments: Mineral composition and other properties of samples.
- Communication signals: Commands and information exchanged with Earth.

2) Characterize the operating environment.

The Mars Rover operates in the following environment:

Property	Characterization	Explanation
Observability	Partially observable	The rover cannot directly observe every environmental condition.
Agents	Single-agent	It primarily performs its mission independently.
Determinism	Stochastic	Terrain conditions and equipment behaviour can be uncertain.
Episodic/Sequential	Sequential	Each action can affect future actions and mission progress.
Dynamics	Dynamic	Environmental conditions may change over time.
State space	Continuous and discrete	Position and temperature are continuous, while action choices are discrete.
Knowledge	Partially unknown	Some terrain and geological properties are initially unknown.

3) What are the actions the agent can take?

The Mars Rover can perform the following actions:

1. Move forward, backward, and turn.
2. Capture images using cameras.
3. Detect and avoid obstacles.
4. Collect soil and rock samples.
5. Analyse samples using scientific instruments.
6. Store scientific data.
7. Transmit data to Earth.
8. Recharge or conserve power when possible.
9. Select a safe route to reach a target location.

4) How can one evaluate the performance of the agent?

The rover's performance can be evaluated using the following criteria:

- Scientific accuracy: Accuracy of sample analysis and scientific observations.
- Exploration coverage: Amount of useful terrain explored.
- Navigation efficiency: Ability to reach target locations using safe and efficient routes.
- Safety: Avoidance of obstacles and prevention of equipment damage.
- Energy efficiency: Effective use of limited battery power.
- Mission completion: Achievement of scientific objectives.
- Communication reliability: Successful transmission of collected data to Earth.

5) What sort of agent architecture is most suitable? Why?

A model-based, goal-based, utility-based learning agent architecture is suitable for a Mars Rover.

Explanation:

- Model-based: Maintains an internal representation of terrain, obstacles, and rover status.
- Goal-based: Plans actions to achieve objectives, such as reaching a rock formation or collecting samples.
- Utility-based: Selects actions that balance scientific value, safety, time, and energy consumption.
- Learning agent: Uses experience and collected data to improve navigation and decision-making.

A hybrid intelligent agent combining these capabilities is suitable because the Mars Rover operates in an uncertain environment and must make autonomous decisions while achieving scientific goals safely and efficiently.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such

that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer:
Problem: Place eight queens on an 8×8 chessboard so that no two queens attack each other.

A queen can attack another queen if both are in the same row, column, or diagonal.

A. Problem formulation

1. Initial state: An empty 8×8 chessboard.

2. State space: All possible valid and partial arrangements of queens on the chessboard.

3. Actions: Place a queen in a safe square in the next column.

4. Transition model: The selected square is occupied by a queen, producing a new board configuration.

5. Goal test: Check whether eight queens have been placed and no two queens share a row, column, or diagonal.

6. Path cost: Each queen placement has a cost of 1. A complete solution requires eight placements, giving a total cost of 8.

B. Search strategies

1. Breadth-First Search (BFS)

- Explores configurations level by level.
- Considers all configurations with the same number of queens before moving to the next level.
- Can find a solution but requires substantial memory.

2. Depth-First Search (DFS)

- Places a queen in a safe square and continues to the next column.
- Backtracks when no safe square is available.
- Uses less memory than BFS.

3. Backtracking search

- Places one queen in each column.
- Checks whether the position is safe before proceeding.
- If a conflict occurs, removes the queen and tries another position.
- Avoids exploring many invalid configurations.

Backtracking is particularly suitable for the Eight Queens problem.

C. Pseudocode

```
def solve_queens(board, col):
    if col == 8:
        return True

    for row in range(8):
        if is_safe(board, row, col):
            board[row][col] = 1

            if solve_queens(board, col + 1):
                return True

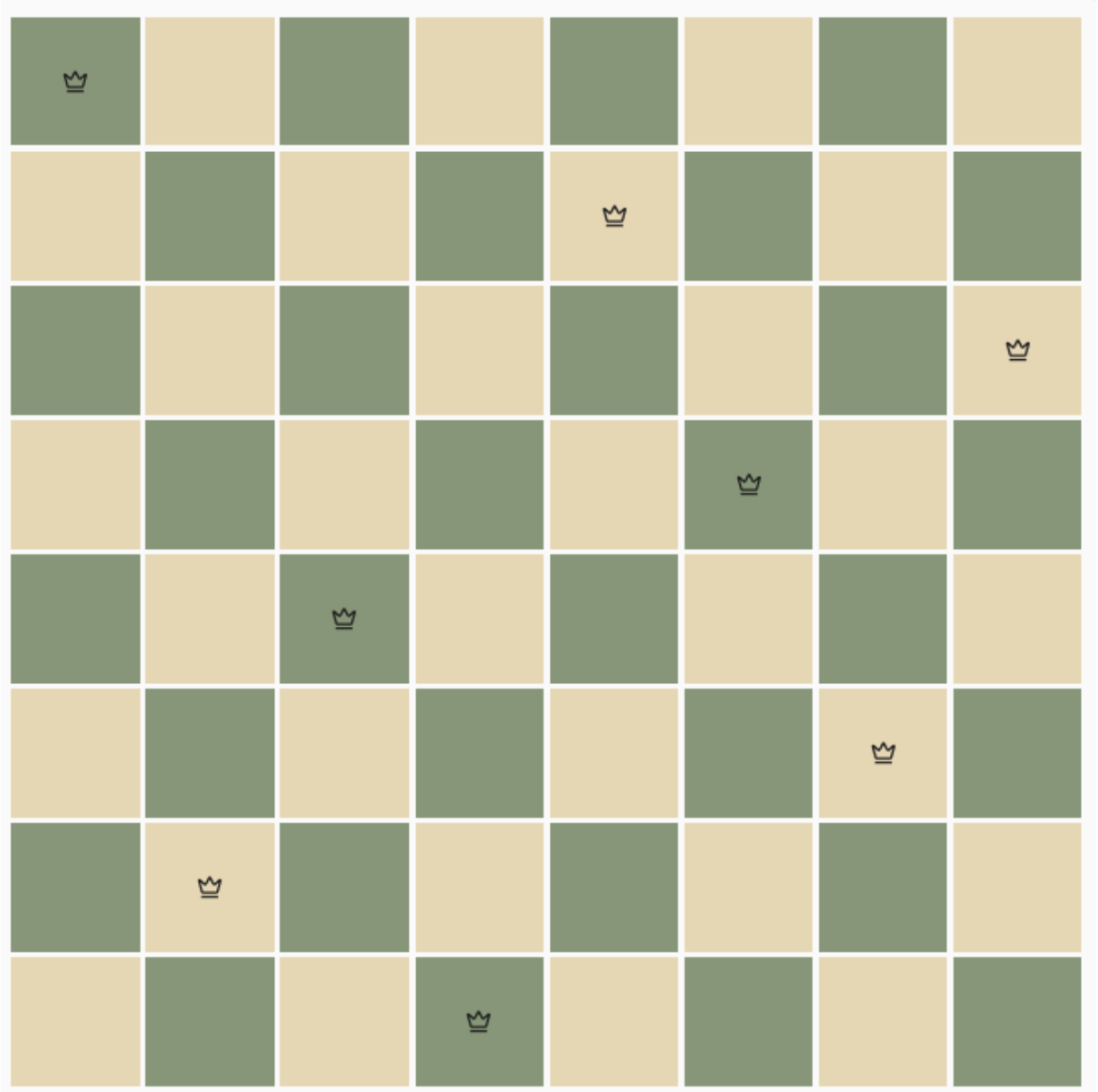
            board[row][col] = 0

    return False
def is_safe(board, row, col):
    for c in range(col):
        if board[row][c] == 1:
            return False
    r, c = row - 1, col - 1
    while r >= 0 and c >= 0:
        if board[r][c] == 1:
            return False
        r -= 1
        c -= 1
    r, c = row + 1, col - 1
    while r < 8 and c >= 0:
        if board[r][c] == 1:
            return False
        r += 1
        c -= 1
    return True
board = [[0] * 8 for _ in range(8)]
if solve_queens(board, 0):
    for row in board:
        print(row)
```

D. Example of a valid solution

One valid arrangement uses these row positions for columns 1 through 8:

$$[1, 5, 8, 6, 3, 7, 2, 4]$$



Conclusion: The Eight Queens problem is a constraint-satisfaction problem. Backtracking search efficiently finds a valid arrangement by placing queens one at a time and rejecting positions that violate the constraints.

4.A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer:

Problem: A customer wants to travel from one location to another using the OLA Cab Booking mobile application. The customer can select a cab type such as Mini, Micro, Sedan, Shared, or Prime.

A. Type of problem-solving agent

A goal-based problem-solving agent is suitable for this problem.

The agent's goal is to transport the customer from the source location to the destination while considering the selected cab type, estimated fare, travel time, and availability.

A utility-based agent can also be used when the system must compare multiple options and recommend the best cab based on cost, comfort, waiting time, and customer preferences.

B. Problem formulation

1. Initial state: Customer is at the pickup location.
2. Goal state: Customer reaches the destination.
3. Actions: Enter locations, select a cab type, check availability, book a cab, and follow the route.
4. Transition model: Each action updates the booking or journey state.
5. Goal test: Verify that the customer has reached the destination.
6. Path cost: Estimated fare, travel time, distance, and waiting time.

C. Pseudocode for OLA Cab Booking

```
BEGIN

  INPUT pickup_location
  INPUT destination

  IF pickup_location = destination THEN
     DISPLAY "Pickup and destination must be different"
     STOP
  ENDIF

  DISPLAY "Select Cab Type"
  DISPLAY "1. Mini"
  DISPLAY "2. Micro"
  DISPLAY "3. Sedan"
  DISPLAY "4. Shared"
  DISPLAY "5. Prime"

  INPUT cab_type

  CHECK cab availability

  IF cab is unavailable THEN
     DISPLAY "Cab not available"
     STOP
  ENDIF

  CALCULATE estimated_fare
  CALCULATE estimated_travel_time
```

```
    DISPLAY estimated_fare
    DISPLAY estimated_travel_time

    INPUT customer_confirmation

    IF customer_confirmation = YES THEN
        BOOK cab
        ASSIGN driver
        PLAN route
        DISPLAY "Cab booked successfully"

        WHILE destination not reached
            TRACK cab location
            UPDATE route if required
        ENDWHILE

        DISPLAY "You have reached your destination"
    ELSE
        DISPLAY "Booking cancelled"
    ENDIF

END
```

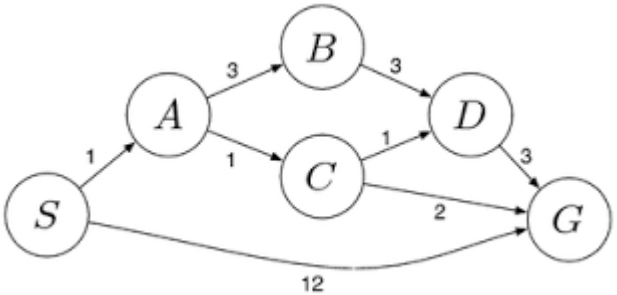
D. Explanation

1. The customer enters the pickup and destination locations.
2. The system displays available cab types.
3. The customer selects a preferred cab.
4. The system checks cab availability and calculates the estimated fare and travel time.
5. After customer confirmation, the system books a cab and assigns a driver.
6. The route is planned and tracked during the journey.
7. When the destination is reached, the goal is achieved.

A goal-based problem-solving agent helps the OLA application achieve the customer's travel goal. A utility-based extension can compare available cabs and recommend an option that best matches the customer's preferences.

5.A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Problem statement

A logistics company wants to find the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

UCS expands the node with the lowest cumulative path cost first. It considers the total cost from the starting node, rather than just the cost of the next edge.

1. Given weighted graph

The routes and their costs are:

Route	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

The initial node is S, and the goal node is G.

2. Step-by-step execution of UCS

Step 1: Start at S

The initial path cost is $g(S) = 0$.

Expand S and add its neighboring nodes to the frontier.

- S → A = 0 + 1 = 1
- S → G = 0 + 12 = 12

Frontier: A(1), G(12)

Step 2: Expand A

A has the lowest cost, 1. Expand A and calculate the costs of its neighbors.

- S → A → B = 1 + 3 = 4
- S → A → C = 1 + 1 = 2

Frontier: C(2), B(4), G(12)

Step 3: Expand C

C has the lowest cost, 2. Expand C.

- S → A → C → D = 2 + 1 = 3
- S → A → C → G = 2 + 2 = 4

Frontier: D(3), B(4), G(4), G(12)

The direct route S → G with cost 12 is more expensive than the newly discovered route to G with cost 4, so the better route is retained.

Step 4: Expand D

D has the lowest cost, 3. Expand D.

- S → A → C → D → G = 3 + 3 = 6

The existing route to G with cost 4 is better than this route with cost 6.

Frontier: B(4), G(4), G(6), G(12)

Step 5: Select the next lowest-cost node

B and G both have cumulative cost 4. If B is selected first, its route to D has cost $4 + 3 = 7$, which is worse than the existing cost of 3 for D. Therefore, no better route is found.

Next, G is selected with cost 4. Since G is the goal node, UCS terminates.

3. UCS expansion table

Step	Node expanded	Cumulative cost	Frontier after expansion
1	S	0	A(1), G(12)
2	A	1	C(2), B(4), G(12)
3	C	2	D(3), B(4), G(4), G(12)
4	D	3	B(4), G(4), G(6), G(12)
5	B	4	G(4), G(6), G(12)
6	G	4	Goal reached; stop

Note: B and G have equal costs at step 5. This table assumes B is selected first to break the tie.

4. Final least-cost path

Optimal route

S

A

C

G

Total cost calculation

$$ 1+1+2=\boxed{4} $$

Minimum cost = 4

Uniform Cost Search finds the least-cost path by expanding nodes in increasing order of cumulative path cost. For the given graph, the optimal path is S → A → C → G, with a total cost of 4. UCS guarantees an optimal solution when all edge costs are non-negative.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem